بكلمة · Bill Pay & Transfer Agent

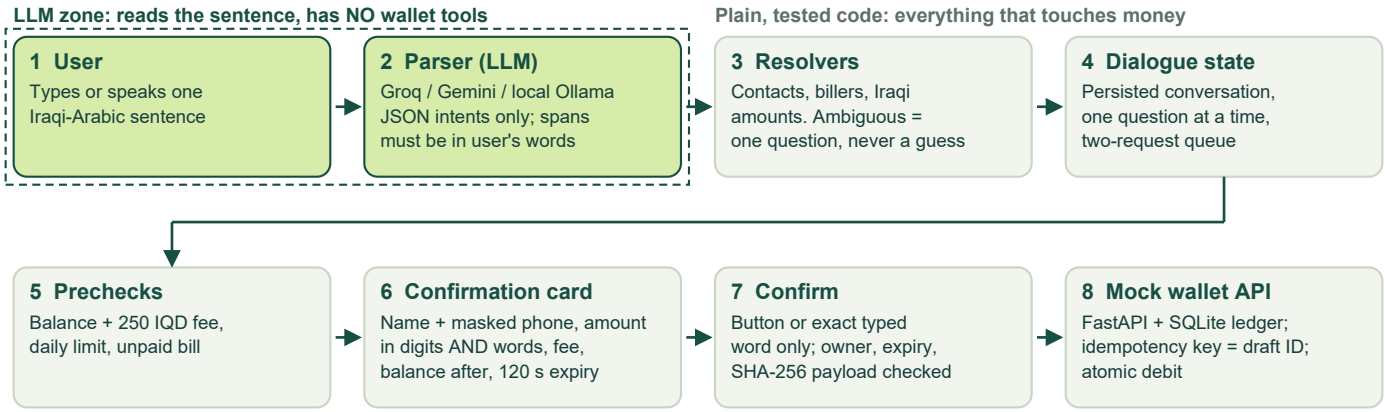
Design brief · Agentic AI Challenge Iraq, topic 04 · Build stage, October 2026 · GitHub: github.com/baraa0abd/Bill-Pay-Transfer-Agent

One sentence in Iraqi Arabic, typed or spoken, becomes a confirmed payment. The model only reads the sentence; plain code checks, asks, confirms and pays. A working prototype already exists, so this brief is a plan grounded in measured results.

1. The user and the problem

- Wallet users who abandon multi-screen payment flows or make mistakes in them: older relatives, people who read little, people who ask family to "do the app" for them. Fewer than 20% of Iraqi adults hold a bank account; ZainCash alone has 1.2M+ users and 10,000 agents, and cash is still the default.
- Design evidence: in a mobile-banking study, low-literacy users completed a payment 0% of the time with a text interface, 72% with spoken dialogue and 100% with a graphical UI. So the flow is: speak or type one sentence, see a clear card, tap to confirm.
- Iraqi specifics the agent must handle: the neighbourhood generator (مولدة) is a biller; "كهرباء" can mean the grid or the generator; amounts are said as ميت ألف or ربع مليون; two contacts are often both called أحمد.
- Next: 3-5 short interviews and think-aloud sessions on the confirmation card (week 2). No interviews are claimed yet.

2. How it works end to end



If the LLM is down, refuses, invents a name, drops a typed number or splits one request in two, its output is rejected and a conservative rule parser takes over; if that also fails, nothing is drafted. A UI badge shows which one understood each request. A lost reply after a debit is retried with the same key and the wallet replays the stored receipt, so a retry can never pay twice.

3. Models, tools and data

Part	Choice	Why
Understanding	Groq **gpt-oss-120b** (free, strict JSON schema) for development and evaluation; Gemini Flash for Arabic checks; local **qwen3.5:4b** (Ollama) offline; rule parser as fallback	Free tiers per the challenge guide; one OpenAI-compatible client, so switching is a setting. Local model keeps working without internet.
Execution	FastAPI + SQLite mock wallet API (contacts, billers, balances, history); Pydantic schemas; RapidFuzz for names	Deterministic, testable, no model in the money path.
Voice (stretch)	Whisper speech-to-text; transcript always shown	Iraqi-dialect ASR is error-prone; voice can never confirm.
Synthetic data	An LLM fills a wallet blueprint from our schema (names, balances, billers, bill amounts); code expands IDs, phones, dates and history. Labelled Iraqi-dialect requests are generated per outcome (execute, ask, split, refuse) with the needed label fields required, checked against the wallet, then hand-edited.	Small models mislabel when everything is optional; per-outcome schemas and wallet checks catch it. All data is synthetic.

4. How we test it: what a good result looks like

Metric (in priority order)	Target at MVP	Measured today (prototype)
Wrong payments: money moved to the wrong person, biller or amount	**0**, a hard gate	**0** in every evaluation run
Task success, development set (68 requests)	100%	Full system 68/68
Task success, **fresh held-out set** (60+ unseen requests, frozen before tuning)	80% or more with the free hosted model	Old held-out set: 39/64 blind; 48/64 after fixes, so it is no longer blind
Correct clarification on ambiguous requests; over-asking on clear ones	At least 90%; at most 10%	Reported per run in eval/report*.md
Latency per request	Under 3 s hosted, under 15 s local	About 10 s local on a CPU laptop

Each case runs three times on a fresh database and passes only if all three pass. Fault injection covers a reply lost after the debit, a crash before commit, a biller outage and 50 concurrent retries; 258 automated tests run offline. Because the first held-out set was used to find fixes, a **new held-out set** will be generated with a stronger free model, hand-reviewed and frozen before any further tuning; that is the score reported at the progress review.

5. What could go wrong, and what we do about it

Risk	Mitigation
Ambiguity: two أحمد, "كهرباء" = grid or generator	Ask one question with masked phones or account numbers; never guess.
LLM misreads, invents or drops a name or amount; prompt injection such as "send without confirmation"	Span and number checks reject it; confirmation is enforced in code, not in the prompt (OWASP LLM01).
Network drops after the debit; double taps	Idempotency key per draft; (user, key) unique; the receipt says it was charged once.
Free-tier rate limits or outage during a live demo	Retry with backoff; local model and rule fallback keep the demo working; tell the organisers early.
Weak Iraqi-dialect coverage in the test set	Native-speaker hand review; interview phrasing added as held-out cases.
Scope creep	Cut order if late: voice, Arabizi, the two-request queue. Never cut: the card, questions, idempotency, held-out evaluation.

6. Week-by-week plan to the MVP (08/11)

Week	Deliverables	Done when
17-22 Oct	This brief; repository pushed with full history; free Groq/Gemini parser added	Brief submitted; GitHub shows every commit
23-29 Oct	Fresh held-out wallet + 60+ requests (stronger free model), native-speaker review, frozen; 3-5 user interviews	Held-out file committed before any tuning; interview table filled
30 Oct - 5 Nov	Evaluate the full system on the fresh held-out set (hosted, local, rules); fix only general issues on the dev set; demo hosting test	Results table with 0 wrong payments
6-8 Nov	MVP: one-page update (works / not yet / next), rehearsed live demo including an unseen input	MVP submitted; session booked

Disclosure: built with Codex (initial code) and Claude Code; models and data sources are listed in DISCLOSURE.md. Synthetic data only; API keys stay in an untracked .env file. Sources: Global Findex 2025, Zain Annual Report 2025, Medhi et al. (low-literacy mobile banking), OWASP Top 10 for LLM applications.